SESSION DE RATTRAPAGE – JEE1 ET JEE2

Document d'explication

Conception et fonctionnement de l'application de facturation

Clavstel El Schaddai BOYAOU

Classe : M1 GELO

TP2 : Spring Boot + React + MongoDB

Année universitaire 2025-2026

1. Introduction

Ce document explique la conception et le fonctionnement de l'application de facturation réalisée pour le TP2 de JEE. Elle reprend l'architecture du TP1 (Spring Boot) en remplaçant le frontend HTML/CSS par **React** et la base MySQL par **MongoDB**. Le gérant gère les clients et les factures ; le client peut se connecter pour consulter ses factures.

2. Cahier des charges

- Entités : Facture (Idfact, montant, objet, IdClient), Client (Idclient, nom, tel), Gérant (Id, nom, login, password).
- Gérant : authentification, ajout, modification, suppression et recherche de clients et de factures.
- Client : connexion puis consultation de ses factures (nouveauté du TP2).
- Technologies : Spring Boot, React, MongoDB.

Pour permettre la connexion du client, deux champs ont été ajoutés à l'entité Client : **login** et **password** (chiffré). Une **date** d'émission a aussi été ajoutée aux factures.

3. Architecture



Figure 1 – Architecture générale

Le projet est découpé en deux applications indépendantes : un **backend** (API REST, port 8080) et un **frontend** (React, port 5173). Elles communiquent en JSON via HTTP. Cette séparation est le principal changement par rapport au TP1, où les pages HTML étaient générées côté serveur.

Organisation du backend (package com.facturation)

Package	Rôle
model	Documents MongoDB : Gerant, Client, Facture et FactureView (facture + nom du client)
repository	Interfaces Spring Data MongoDB (CRUD et requêtes dérivées)
controller	Endpoints REST : AuthController, ClientController, FactureController
security	JwtService (création / lecture du jeton) et JwtFilter (authentification de chaque requête)
config	SecurityConfig (règles d'accès, CORS, BCrypt), GlobalExceptionHandler, DataInitializer

4. Passage de MySQL à MongoDB

En MongoDB, les données sont stockées en documents JSON regroupés en collections (équivalent des tables). Il n'y a pas de clé étrangère : la facture conserve simplement l'identifiant du client dans le champ **idClient**. Spring Data MongoDB permet d'écrire les accès aux données sans SQL : par exemple `findByIdClient(String)` est transformé automatiquement en requête.



Figure 2 – Collections MongoDB

Points d'attention

- La suppression d'un client supprime ses factures (`deleteByIdClient`) car MongoDB n'applique pas de cascade automatique.
- L'identifiant `_id` est un ObjectId généré par MongoDB, manipulé comme String dans Java.
- Les mots de passe ne sont jamais stockés en clair : hash BCrypt.

5. Sécurité

L'authentification repose sur des **jetons JWT** (JSON Web Token) signés avec HMAC-SHA. L'application est donc **sans état** : aucune session n'est conservée sur le serveur.

- **Connexion :** `POST /api/auth/login` compare le mot de passe saisi au hash BCrypt. Le serveur cherche d'abord un gérant, puis un client, et renvoie le jeton, le rôle et le nom.
- **Requêtes suivantes :** React ajoute l'en-tête `Authorization: Bearer <jeton>`. JwtFilter vérifie la signature et l'expiration (2 h), puis place le rôle dans le contexte de sécurité.
- **Autorisations** (SecurityConfig) : /api/auth/** public ; /api/factures/mes-factures réservé au rôle CLIENT ; le reste de /api/** réservé au rôle GERANT.
- **Isolation des données :** pour « mes factures », l'identifiant du client est lu dans le jeton, jamais dans l'URL : un client ne peut pas consulter les factures d'un autre.
- **CORS :** seule l'origine http://localhost:5173 est autorisée à appeler l'API.

6. Détail des fonctionnalités

6.1 Gestion des clients (ClientController)

- **Rechercher :** GET /api/clients?q=… filtre par nom (insensible à la casse) ou téléphone.
- **Ajouter :** validation du nom et du login, mot de passe obligatoire, login unique (parmi gérants et clients), mot de passe chiffré avant enregistrement.
- **Modifier :** mise à jour des champs ; le mot de passe n'est changé que s'il est renseigné.
- **Supprimer :** supprime d'abord les factures du client puis le client.

6.2 Gestion des factures (FactureController)

- **Rechercher :** GET /api/factures?q=… filtre par objet, nom du client ou numéro.

- **Ajouter / modifier :** validation (montant positif, objet non vide) et vérification que le client existe ; la date du jour est ajoutée par défaut.
- **Supprimer :** suppression par identifiant.
- **Espace client :** GET /api/factures/mes-factures renvoie les factures du client connecté.

6.3 Gestion des erreurs

Les erreurs sont renvoyées au format `{"message": "..."}` (400 validation, 401 non authentifié, 403 accès refusé, 404 introuvable, 409 login déjà utilisé). Le frontend affiche ce message dans un bandeau rouge.

7. Frontend React

Fichier	Rôle
main.jsx	Point d'entrée : monte App dans BrowserRouter
App.jsx	Routes et composant Protected (vérifie la session et le rôle, sinon redirige vers /login)
Layout.jsx	Barre de navigation adaptée au rôle et bouton Déconnexion
api.js	Fonctions d'appel de l'API (fetch) ; ajoute le jeton ; gère les erreurs et le 401
pages/LoginPage.jsx	Formulaire de connexion, stockage du jeton dans localStorage
pages/ClientsPage.jsx	Liste, recherche, formulaire d'ajout / modification, suppression des clients
pages/FacturesPage.jsx	Idem pour les factures, avec liste déroulante des clients
pages/MesFacturesPage.jsx	Espace client : factures et total

React utilise des **hooks** : `useState` conserve l'état (liste, formulaire, message d'erreur) et `useEffect` charge les données au affichage de la page. Toute modification (ajout, suppression…) est suivie d'un rechargement de la liste.

8. Déroulement d'une requête (exemple : ajouter une facture)

- Le gérant remplit le formulaire et clique sur Enregistrer.
- FacturesPage appelle `api.factures.create()` : requête POST /api/factures avec le jeton.
- JwtFilter authentifie la requête ; SecurityConfig vérifie le rôle GERANT.
- FactureController valide les données, vérifie le client, enregistre dans MongoDB.
- Le serveur répond 201 avec la facture ; React ferme le formulaire et recharge la liste.

9. Mise en route et tests

Voir le guide d'utilisation pour l'installation. Scénarios de test recommandés : connexion gérant / client, mauvais mot de passe (erreur), ajout d'un client puis connexion avec son compte, ajout d'une facture, recherche, modification, suppression d'un client (ses factures disparaissent), tentative d'un client d'accéder à /gerant/clients (redirection).

10. Limites et améliorations possibles

- Pas de tests automatisés (JUnit / Testing Library) : à ajouter.

- Jeton stocké dans localStorage : une solution à cookie HttpOnly serait plus robuste en production.
- Secret JWT et mot de passe du gérant par défaut à changer avant tout déploiement.
- Pas de pagination : à prévoir pour de gros volumes.
- Idées : statut payé / impayé, export PDF d'une facture, tableau de bord de statistiques, déploiement Docker.